



**Universidade Federal de Ouro Preto
Instituto de Ciências Exatas e Aplicadas
Departamento de Computação e Sistemas**

Um Simulador de maquinas de estados

Matheus Henrique Vieira Ribeiro

TRABALHO DE CONCLUSÃO DE CURSO

ORIENTAÇÃO:

Elton Máximo Cardoso

COORIENTAÇÃO:

Nome Completo do Coorientador

**Fevereiro, 2024
João Monlevade–MG**

Matheus Henrique Vieira Ribeiro

Um Simulador de maquinas de estados

Orientador: Elton Máximo Cardoso

Coorientador: Nome Completo do Coorientador

Monografia apresentada ao curso de Sistemas de Informação do Instituto de Ciências Exatas e Aplicadas, da Universidade Federal de Ouro Preto, como requisito parcial para aprovação na Disciplina “Trabalho de Conclusão de Curso II”.

Universidade Federal de Ouro Preto

João Monlevade

Fevereiro de 2024

A Ficha Catalográfica é elaborada exclusivamente pela Biblioteca. Substitua esta página pelo documento gerado na versão final da sua monografia.

A Folha de aprovação deverá ser gerada pelo(a) professor(a) orientador(a) após a realização das correções sugeridas pela banca examinadora.

As instruções para elaboração desse documento eletrônico estão disponíveis em <<https://www.monografias.ufop.br>> (menu “Documentos”, opção “Tutorial Professor Orientador”).

Após gerar a folha de aprovação, o(a) professor orientador(a) enviará o documento ao(à) orientado(a), o qual deverá inseri-lo nesta página.

Este trabalho é dedicado ...

Agradecimentos

Agradeço ...

“Science is more than a body of knowledge; it is a way of thinking.”

— Carl Sagan (1934 – 1996),
in: The Demon-Haunted World: Science as a Candle in the Dark.

Resumo

Segundo a ??, 3.1-3.2), o resumo deve ressaltar o objetivo, o método, os resultados e as conclusões do documento. A ordem e a extensão destes itens dependem do tipo de resumo (informativo ou indicativo) e do tratamento que cada item recebe no documento original. O resumo deve ser precedido da referência do documento, com exceção do resumo inserido no próprio documento. (...) As palavras-chave devem figurar logo abaixo do resumo, antecidas da expressão Palavras-chave:, separadas entre si por ponto e finalizadas também por ponto.

Palavras-chaves: latex. abntex. editoração de texto.

Abstract

This is the english abstract.

Key-words: latex. abntex. text editoration.

Lista de ilustrações

Figura 1 – a Criação do estado, b Editando o estado	25
Figura 2 – a Criação da transição, b Editando a transição	25
Figura 3 – Um APN simples	26
Figura 4 – entrada de palavra 0101	26
Figura 5 – a tabela de caracteres, b Visualização dos snapshots	27
Figura 6 – a caminho de aceitação, b clique em snapshot, c botão de proximo . . .	28
Figura 7 – a botões de configuração, b botões de download	29

Lista de tabelas

Tabela 1 – Trabalhos relacionados	19
---	----

Lista de abreviaturas e siglas

Lista de símbolos

Γ	Letra grega Gama
Λ	Lambda
ζ	Letra grega minúscula zeta
$\in$	Pertence

Sumário

1	INTRODUÇÃO	14
1.1	Elaboração do capítulo	14
1.2	O problema de pesquisa	14
1.3	Objetivos	15
1.4	Metodologia	15
1.5	Organização do trabalho	15
2	REVISÃO BIBLIOGRÁFICA	17
2.1	Trabalhos correlatos	17
2.2	Derivadas	19
2.3	Maquinhas de estados	20
3	DESENVOLVIMENTO	22
3.1	Implementação	22
3.1.1	Representação lógica	22
3.2	Uso do software	24
4	RESULTADOS	30
5	CONCLUSÃO	31
	REFERÊNCIAS	32

1 Introdução

A teoria da computação estuda as propriedades dos sistemas computacionais. Esta teoria foi formalizada pela primeira vez, de forma efetiva, em 1936, por Alan Turing e Alonzo Church, que construíram as bases do que hoje chamamos de computação, por meio da elaboração do conceito de linguagens formais.

Linguagens formais são lidas por meio de máquinas de estados e escritas por meio de gramáticas formais. Neste trabalho, serão enfatizadas as máquinas de estados, mais especificamente autômatos de pilha não determinísticos (APN), por serem particularmente mais abstratos e de representação mais complexa que as demais máquinas de estados.

Sendo a teoria da computação a base das tecnologias atuais, torna-se essencial que os discentes dos cursos da área de computação, como sistemas de informação e engenharia da computação, dominem esses conceitos para que se tornem bons profissionais. Uma vez que tais conceitos não apenas fornecem a base teórica para a computação, mas também promovem habilidades de pensamento crítico e análise rigorosa, conferindo a capacidade de formalizar problemas, definir algoritmos e analisar sua eficiência.

Como forma de auxiliar os discentes dos cursos da área da computação a visualizar de forma mais clara os autômatos de pilha não determinísticos, este trabalho tem como objetivo desenvolver uma ferramenta web para montagem e visualização de APNs em execução.

1.1 Elaboração do capítulo

Este capítulo apresenta o seu trabalho. Você deve contextualizar o problema abordado, descrever os objetivos gerais e específicos, apresentar a metodologia e como o trabalho está estruturado.

1.2 O problema de pesquisa

Ciente da importância do estudo da teoria da computação, utilizada nos dias de hoje como ferramenta para o desenvolvimento dos compiladores das linguagens de programação modernas, é fundamental que os alunos dos cursos superiores da área de computação dominem esta disciplina. Contudo, eles normalmente precisam lidar com alguns desafios durante o estudo, sendo dois deles:

- O estudo das máquinas de estados é realizado utilizando apenas diagramas em um

papel e abstrair o seu funcionamento dada essa representação limitada pode se tornar um desafio;

- Criar uma maquina de estado que reconheça uma linguagem especifica e apenas ela, é como responder uma charada ou enigma, em que uma resposta precisa satisfazer uma serie de condições especificas para ser considerada correta.

Este trabalho visa amenizar o primeiro desafio e auxiliar no enfrentamento do segundo, facilitando o exercício pratico da disciplina.

1.3 Objetivos

O presente trabalho consiste em facilitar o aprendizado da disciplina de fundamentos teóricos da computação.

Este trabalho possui aos seguintes objetivos específicos:

- Criar um software gráfico que auxilie os alunos simulando o funcionamento de maquinas de estado em tempo real;
- Disponibilizar esse sistema para alunos da disciplina de fundamentos teóricos da computação do campus ICEA;
- Avaliar o impacto do sistema no aprendizado dos alunos por meio de um questionário.

1.4 Metodologia

O objeto de pesquisa deste trabalho ...

Os passos para execução deste trabalho são assim definidos:

- Revisão da literatura
- Desenvolvimento do software de auxilio aos alunos
- Teste por parte dos alunos
- Análise do feedback dos alunos

1.5 Organização do trabalho

É importante observar que a estrutura é apresentada a partir do próximo capítulo. O capítulo de Introdução não deve compor esta descrição. Além disso,

sempre que você fizer referência à algum item específico, a inicial deve ser maiúscula. Por exemplo, Capítulo 2, Tabela 5, Figura 1, dentre outros.

O restante deste trabalho é organizado como se segue. O Capítulo [2](#) apresenta...

2 Revisão bibliográfica

2.1 Trabalhos correlatos

Foram encontrados 10 sistemas com Objetivos similares ao proposto por este projeto, ao longo desta sessão cada um deles será exposto, podendo ser visualizado uma comparação simples dos principais pontos de cada um por meio da tabela 1

O primeiro foi o Language emulator ([VIEIRA; VIEIRA; VIEIRA, 2003](#)), projeto desenvolvido em 2003 na UFMG utilizando a linguagem de programação java pelos pesquisadores Luiz Filipe Menezes Vieira, Marcos Augusto Menezes Vieira e Newton José Vieira. O projeto se trata de um simulador de autômatos que tem foco nas principais operações envolvendo autômatos finitos determinísticos, autômatos finitos não determinísticos com e sem transição lambda, maquinas de Moore e maquinas de Mealy, juntamente com as gramáticas regulares e as expressões regulares. As operações realizadas por esse sistema incluem transformações entre diferentes tipos de maquinas de estados e gramatica, assim como a capacidade de determinar de uma palavra pertence ou não a essa gramatica. Apesar de ser um projeto bastante completo em relação as suas funcionalidades, ele não apresenta uma interface que detalhe com diagramas essas operações, sendo interativa apenas por meio de botoes e textos.

O segundo sistema foi AUTOMATOGRAPH ([KIENETZ; CANAL, 2004](#)), desenvolvido por Eduardo Bacchi Kienetz e Ana Paula Canal na UFRGS no ano de 2006 utilizando a linguagem de programação java e consegue reconhecer palavras de uma linguagem regular pertencentes a linguagem de um automato informado pelo usuário e gerar uma gramatica regular a partir desse automato, porém assim como a anterior não possui uma interface gráfica que represente o diagrama desses autômatos.

Em seguida Ginix Abstract Machine (GAM) ([JUKEMURA; NASCIMENTO; UCHÔA, 2005](#)), que é um sistema desenvolvido por Anibal Jukemura, Hugo Nascimento e Joaquim Uchôav na UFLA no ano de 2005, tem a capacidade de representar em forma de grafos direcionados AFDs, AFNs, AFNLs, APDs, APNs e MTs além de reconhecer se uma palavra pertence ou não a maquina de estados representada por esse grafo, a GAM tambem consegue transformar AFNs em AFDs e minimizar AFDs.

FSM Simulator ([ZUZAK; JANKOVIC, 2025](#)), é um aplicativo web desenvolvido por Ivan Zuzak e Vedrana Janković utilizando html, javascript e bootstrap. O FSM Simulator consegue representar o funcionamento e criar aleatoriamente AFDs, AFNs e expressões regulares em regex. além de criar e palavras aleatórias e realizar testes. Sua representação se dá por meio de dígrafos estáticos.

AutoSim ou Automaton Simulator ([BURCH, 2008](#)) é um software desenvolvido em java por Carl Burch na Universidade Carnegie Mellon no ano de 2006. AutoSim é capaz de construir e simular de forma dinâmica AFDs, AFNs, APDs e MTs por meio de grafos direcionados.

Java Finite Automata Simulation Tool ou jFAST ([WHITE; WAY, 2006](#)) é um software que foi desenvolvido na universidade de villanova por Timothy M. White e Thomas P. Way no ano de 2006 utilizando a linguagem de programação java e trás a proposta de construir diagramas dinâmicos para simular o comportamento de AFDs, AFNs, APDs e máquinas de turing.

Finite State Machine Dsigner(FSMD) é um aplicativo web desenvolvido por Evan Wallace em 2010 com o objetivo de criar com facilidade diagramas que representassem máquinas de estados em geral e portar esses diagramas para vários formatos como png, json e latex, porém não simula seu funcionamento, apenas desenha as máquinas.

Automaton Simulator ([DICKERSON, 2021](#)) é um aplicativo web criado por Kyle Dickerson em 2021 e tem o proposito de simular o comportamento de AFDs, AFNs e APDs por meio de diagramas.

UC Davis Automaton Simulator(UCDAS) ([DOTY, 2020](#)) é uma aplicação web criada em 2020 por David Doty utilizando a linguagem de programação Elm na universidade da califórnia em Davis e se propõe a ser um simulador que demonstra o comportamento de alguns autômatos e gramáticas sendo: AFDs, AFNs, Regex, GLCs e MTs porém não possui uma interface com diagramas, sendo sua interatividade realizada por meio de texto e botões.

Por fim, o sistema mais completo que é o JFLAP ([RENSSELAER; UNIVERSITY, 2018](#)) que teve seu desenvolvimento iniciado em 1990 na Rensselaer por Dan Caugherty e desde então teve a contribuição de vários desenvolvedores utilizando varias tecnologias como c++, java e javascript, tendo o desenvolvimento transferido para Duke University em 1995 e sua ultima atualização até o momento em 2018. Atualmente JFLAP consegue criar e simular AFD, ANF, APD, APN e MT além de transformar cada automato em sua respectiva gramática e transformar autômatos que são equivalentes entre si como AFD em AFN e outras operações como minimizar AFDs. Realizando todas essas funções com diagramas e elementos gráficos que facilitam bastante o entendimento.

Após visitar estes sistemas, foi possível perceber que apenas o simulador GAM simulava a execução de autômatos de pilha não determinísticos; porém, ele não possuía uma visualização satisfatória do não determinismo desse tipo de autômato, pois cada execução paralela possui sua própria pilha, estado atual e leitura de caractere da palavra, sendo difícil visualizar tantos dados simultaneamente de cada uma das execuções paralelas. Portanto, optou-se por criar um sistema que tivesse como um dos pontos principais representar esse

Sistema	ano	linguagem	idioma	automatos	operações	interface
Language Emulator	2003	java	Português	3	sim	apenas texto
AUTOMATO-GRAPH	2004	java	Português	2	não	apenas texto
GAM	2005	-	Português	6	sim	gráfica estática
FSM Simulator	-	javascript	Ingles	2	não	gráfica estática
AutoSim	2006	java	Ingles	4	não	gráfica dinâmica
jFAST	2006	java	Ingles	4	não	gráfica dinâmica
FSMD	2010	javascript	ingles	1	não	gráfica dinâmica
Automaton Simulator	2021	javascript	ingles	3	não	gráfica dinâmica
UCDAS	2020	Elm	ingles	4	não	apenas texto
JFLAP	1990-2018	-	ingles	5	sim	gráfica dinâmica

Tabela 1 – Trabalhos relacionados

não determinismo de forma aprimorada.

2.2 Derivadas

Em seu artigo de 1964 Janusz Brzozowski traz a ideia de derivadas de linguagens formais, utilizando expressões regulares. Derivadas de acordo com Brzozowski surgem ao consumir o primeiro símbolo de uma expressão regular a partir de algumas regras.

$$\begin{aligned}
&\text{Dada uma linguagem } e_1 \\
&\text{se } e_1 = a, \text{ então } d(a, e_1) = \varepsilon \\
&\text{se } e_1 \neq a, \text{ então } d(a, e_1) = \emptyset \\
&d(\varepsilon, e_1) = e_1
\end{aligned}$$

$$\begin{aligned}
&\text{Dada uma linguagem } e^* \\
&d(a, e^*) = d(a, e)e^*
\end{aligned}$$

$$\begin{aligned}
&\delta(e^*) = \varepsilon \\
&\delta(e) = \varepsilon \\
&\delta(\varepsilon) = \emptyset
\end{aligned}$$

$$\begin{aligned}
&\text{Dada uma linguagem } e_1e_2 \\
&d(a, e_1e_2) \\
&d(a, e_1)e_2 | \delta(e_1)d(a, e_2)
\end{aligned}$$

$$\begin{aligned}
&\text{Dada uma linguagem } (e_1|e_2) \\
&d(a, (e_1|e_2)) = d(a, e_1) | d(a, e_2)
\end{aligned}$$

Um exemplo utilizando as regras:

$$\begin{aligned}
&\text{Dada a linguagem } (a|\varepsilon)bc \\
&d(b, (a|\varepsilon)bc) \\
&d(b, (a|\varepsilon))bc | \delta(a|\varepsilon)d(b, bc) \\
&(d(b, a) | d(b, \varepsilon))bc | \varepsilon d(b, bc) \\
&(\emptyset | \emptyset)bc | d(b, bc) \\
&\emptyset | c = c
\end{aligned}$$

Mais tarde em 2011 Matthew Might e David Darais em seu artigo “Parsing with Derivatives” expandiram a ideia das derivadas de Brzozowski para linguagens livres de contexto, de forma que as derivadas são aplicadas sobre o resultado de linguagem já derivadas até a obtenção de gramáticas

Em 2019 mais uma vez essa ideia foi expandida por Ian Henriksen em seu artigo “Derivative grammars: a symbolic approach to parsing with derivatives” de forma que foi otimizada a criação de derivadas gramaticais, excluindo estados inalcançáveis enquanto se mantém regras originais equivalentes e ainda alcançáveis as regras da gramática derivada

2.3 Maquinhas de estados

Máquinas de estado são modelos abstratos que podem ser transdutoras ou reconhecedoras de palavras, sendo as transdutoras aquelas que geram novas palavras pertencentes a uma linguagem específica e as reconhecedoras que reconhecem uma palavra como sendo ou não parte de uma linguagem específica. Neste trabalho daremos foco às máquinas reconhecedoras APD e APN.

Um APD ou automato finito de pilha determinístico é uma máquina de estados formalmente denotada por uma sêxtupla $\langle E, \Sigma, \Gamma, \delta, i, F \rangle$ em que:

- E é conjunto finito de elementos, ditos estados.
- Σ também chamado de alfabeto, é um conjunto finito de elementos distintos, ditos símbolos.
- Γ também chamado de alfabeto da pilha, é um conjunto finito de elementos distintos, ditos símbolos.
- $\delta : E \times \Sigma_\lambda \times \Gamma_\lambda \rightarrow E \times \Gamma_\lambda$

ao AFD de forma que eles podem ser transformados um no outro sem que o seu potencial de reconhecer palavras seja alterado. A diferença entre um AFD e um AFN é o não determinismo classificado pela existência de duas transições de mesmo valor saindo de um mesmo estado para destinos diferentes. AFN λ se trata de uma variação de AFN igualmente equivalente a AFD que pode possuir transições sem nenhum tipo de valor de leitura associado a elas, essas transições são chamadas de transições lambda.

Um APN ou automato finito de pilha não determinístico é uma maquina de estados equivalente ao AFD de forma que eles podem ser transformados um no outro sem que o seu potencial de reconhecer palavras seja alterado. A diferença entre um AFD e um AFN é o não determinismo classificado pela existência de duas transições de mesmo valor saindo de um mesmo estado para destinos diferentes. AFN λ se trata de uma variação de AFN igualmente equivalente a AFD que pode possuir transições sem nenhum tipo de valor de leitura associado a elas, essas transições são chamadas de transições lambda.

O AFD ou autômato finito determinístico é formado por um quártupla $\langle \Sigma, E, \delta, i, F \rangle$ onde :

- Σ : É um conjunto não vazio de elementos distintos ditos símbolos.
- E : É um conjunto não vazio de elementos ditos estados.
- $\delta : E \times \Sigma \rightarrow E$: é uma função de total dita função de transição que mapeia cada possível par de estado por símbolo em um estado.
- $i \in E$: É um estado inicial.
- $F \subseteq E$: É um conjunto de estados finais.

um alfabeto, que são os símbolos que ele lê; e um conjunto de transições, que determinam como ele se move de um estado para outro. Além disso, ele tem um estado inicial, de onde começa, e um ou mais estados finais, que indicam se uma sequência foi ou não aceita.

3 Desenvolvimento

Este capítulo apresenta como a ferramenta foi desenvolvida, e descreve a arquitetura de software utilizada, assim como bibliotecas e técnicas de programação. Este capítulo está organizado da seguinte forma : A seção 3.1 descreve

3.1 Implementação

A ferramenta foi desenvolvida utilizando a linguagem de programação JavaScript, com foco na orientação a objetos, com auxílio de HTML5 e CSS para a sua apresentação visual. A aplicação ocorre completamente no front-end e não possui partes executadas em servidor. Também não foi utilizado nenhum tipo de framework para auxiliar a organização do código. Além dessas ferramentas, foi utilizada a biblioteca JavaScript Cytoscape.

A biblioteca Cytoscape é utilizada para manipulação de desenhos em HTML5, com foco na visualização de grafos, e foi escolhida pois é totalmente open source, e grafos são a forma mais comum de representação de autômatos, utilizada, por exemplo, no livro didático (VIEIRA, 2006). A biblioteca foi utilizada apenas para o desenho dos grafos, tendo toda a parte de simulação de autômatos sido desenvolvida durante este trabalho. O código fonte está disponível no GitHub e o software está hospedado de forma gratuita por meio do GitHub Pages para testes.

O sistema desenvolvido originalmente possuía os 6 tipos de máquinas de estados estudados na disciplina de Fundamentos teóricos da computação na UFOP porém devido ao tempo limitado e escopo abrangente foi tomada a decisão de seguir apenas com a representação de APNs e correção de exercícios, caso interesse essa versão beta pode ser encontrada em <https://matheushvribeiro.github.io/simulador-de-automatos/index.html>.

3.1.1 Representação lógica

Ao desenvolver este projeto, foi feita uma divisão entre a representação lógica e gráfica do APN. A representação lógica do apn e sua execução dentro do sistema acontece por meio de quatro classes principais Transition, APN, Snapshot e APNRunner.

Cada transição foi representada utilizando objetos da classe Transition que possui quatro atributos simples: sym, que é o caractere a ser lido na palavra; stkR, que é o valor que deverá ser desempilhado da pilha; stkW, que é o valor que deverá ser empilhado na pilha; e o atributo state, que é o estado de destino daquela transição.

A classe APN ficou responsável por reunir os estados e transições além de alguns

métodos. Cada representado por um número simples, e o conjunto de estados por um array. Já o conjunto de transições foi representado por um mapa que tem como valor as um array de objetos da classe `Transition` e como chave o número referente ao estado de origem da transição.

A classe `APN` possui o método `delta` que utiliza os atributos do seu mapa de `transitions` para determinar quais `transitions` podem ser atingidas a partir de um estado `s`, um caractere de palavra `chr` e de um topo de pilha `r` e retorna esse conjunto de `transitions` como um array. Esse método percorre cada `transition` que tem sua chave no mapa igual a `s`, já que sua chave no mapa é seu estado de origem, e, em cada uma, verifica se o atributo `sym` é igual a `chr` ou à string vazia e se o atributo `stkR` é igual a `r` ou à string vazia. Se ambas as condições forem satisfeitas simultaneamente, essa transição é considerada validada e adicionada em um array que vai ser retornado ao final do método.

O maior desafio ao representar APNs nesse projeto foi representar o seu não determinismo, dado que existem muitas execuções simultâneas do mesmo autômato. A forma utilizada para representar cada um das execuções simultaneas causadas pelo não determinismo foi a criação da classe `Snapshot`, onde cada `Snapshot` representa um ponto da execução de um dos possíveis caminhos do não determinismo, e vários `Snapshots` podem existir simultaneamente, cada um em um caminho diferente de execução.

Para representar isso, a classe `Snapshot` conta com os seguintes atributos: `state`, que representa o estado em que a execução se encontra; `pos`, que representa a posição do caractere lido na palavra que está sendo executada; e `stack`, que representa a pilha naquele momento de execução.

Para visualização do usuário, é criado um grafo de `snapshots` construído na classe `Graph`. essa classe pode ser utilizada para a criação de grafos de qualquer tipo de objeto JavaScript, sendo composta por dois mapas: um que tem como chave um `id` e como valor um único objeto que representa um nó; e um segundo mapa, que representa as arestas entre os nós utilizando os valores de `id` do primeiro mapa, no qual a chave é o nó de origem e o valor é o nó de destino da aresta.

A classe `APNRunner` é a principal responsável pela execução do autômato, possui o atributo `apn` que é um objeto da classe `APN`, `input` que é a palavra `q` será executada pelo `apn`, `snap` que é um array de objetos da classe `snapshot` e `graph` que é um objeto da classe `Graph`.

O principal método responsável pela execução do autômato é o `step`, que percorre cada um dos `Snapshots` atuais utilizando busca em largura mantendo um array com todas as `Snapshots` mais recentes e verifica as transições válidas a partir deles, utilizando o método `delta` presente na classe `APN`, a ser descrita posteriormente. As transições válidas resultantes são utilizadas para criar novos `Snapshots`, que passam a ser os atuais, finalizando

um step. Adicionalmente, a cada step concluído, os novos snapshots são adicionados ao grafo, havendo uma aresta entre cada snapshot original e os snapshots gerados a partir dele dentro de step.

A execução acontece por meio de sucessivas execuções da função step até que todos os Snapshots atuais já tenham concluído a leitura da palavra a ser computada pelo autômato ou até que chegue a um limite de loops de execução definido pelo usuário por meio da interface gráfica.

3.2 Uso do software

A interface do software pode ser dividida em duas partes: criação e edição de APNs e execução do APN com base em uma palavra. A parte de criação de APNs permite: criar estado; editar nome do estado; defini-lo como final, inicial, ambos ou nenhum; excluir estado; criar transição; editar valores da transição; excluir transição; baixar o APN desenvolvido; importar um APN desenvolvido anteriormente; mover cada elemento do APN; escolher um layout pronto do Cytoscape para o autômato.

Já a parte de execução do APN permite: definir a palavra que será lida; definir o número limite de loops de execução da função step; escolher a condição de aceitação entre FS (estado final e pilha vazia), S (pilha vazia), F (estado final); iniciar e parar a execução do autômato; progredir um step na execução; voltar um step na execução; voltar a execução do início; visualizar os dados completos de cada snapshot individualmente, clicando nele.

Para exemplificar a utilização do software, vamos ao cenário de um autômato simples. Começando com a criação de um autômato, para criar cada estado na ferramenta basta clicar no painel esquerdo do sistema, conforme a Figura 1a. Clicando em um estado com o botão direito, aparecem suas opções de edição do estado, sendo: "inicial", que torna o estado inicial ou faz com que o mesmo deixe de ser inicial; "final", que segue a mesma lógica, tornando o estado final ou deixando de ser; e as opções "renomear" e "excluir", que renomeiam ou excluem o estado do APN. Essas funções podem ser vistas na Figura 1b.

Para criar uma transição, clique no estado de origem e, em seguida, no estado de destino com o botão esquerdo do mouse. A transição padrão tem o valor vazio em todos os elementos, conforme mostrado na Figura 2a. Ao clicar em uma transição com o botão direito, aparecem suas opções de edição, conforme a Figura 2b, sendo: "excluir", que permite excluir a transição, e "renomear", que permite mudar o carácter de leitura, o carácter a ser desempilhado e o carácter a ser empilhado. Um autômato simples construído na ferramenta pode ser visto na Figura 3.

Ao digitar a palavra de leitura no campo escrito "Entrada", como mostrado na

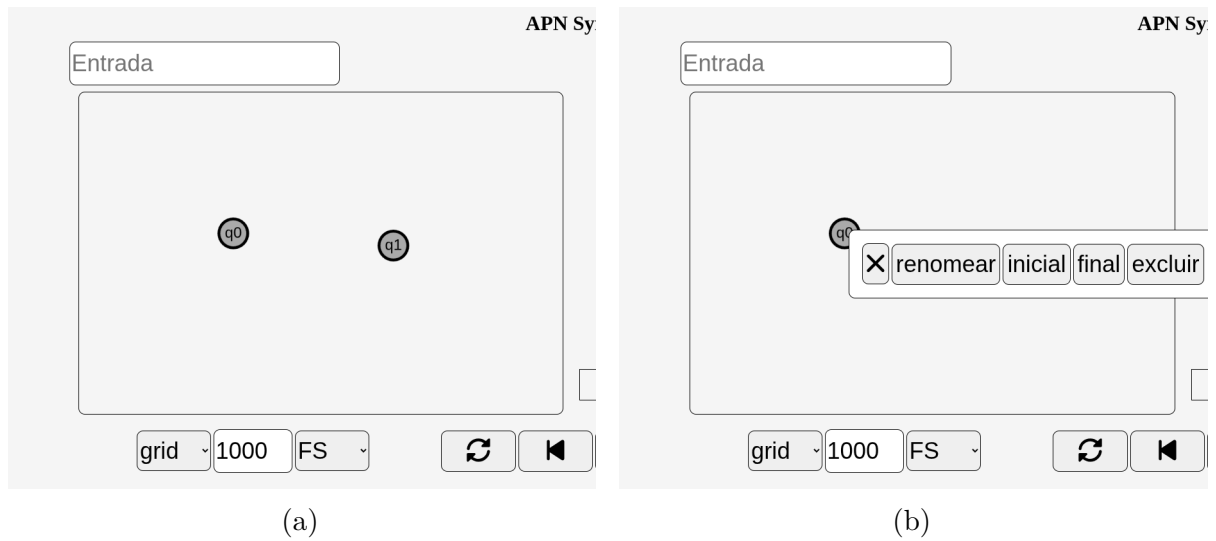


Figura 1 – [a](#) Criação do estado, [b](#) Editando o estado

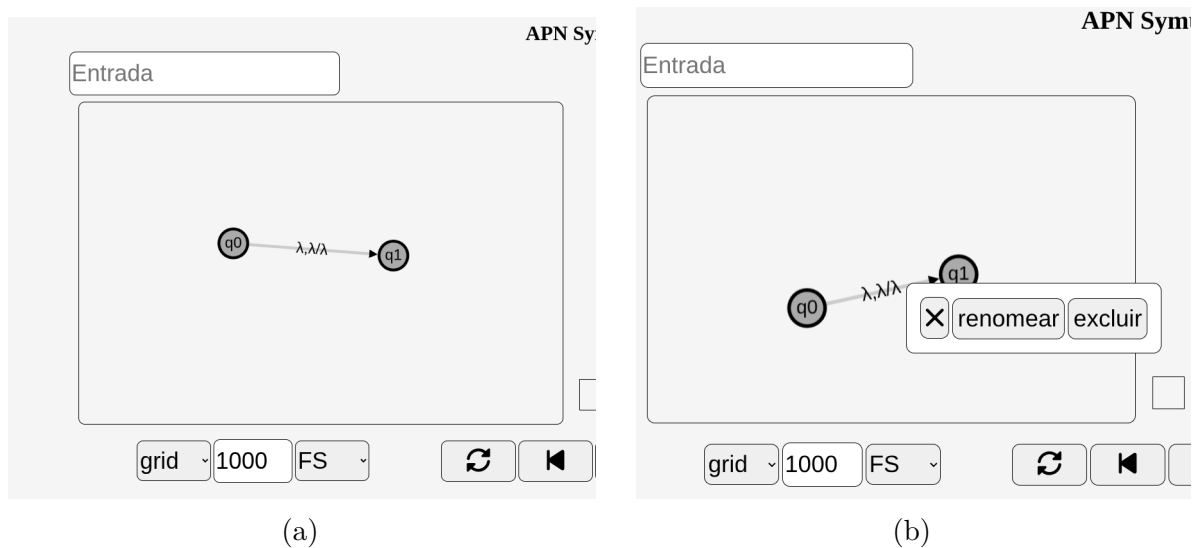


Figura 2 – [a](#) Criação da transição, [b](#) Editando a transição

Figura 4, a palavra "0101", e clicar no botão inferior com ícone play, o sistema calcula a execução completa do autômato de acordo com aquela palavra, e o botão de play tem o ícone modificado para stop, que, quando clicado, permite parar a demonstração do autômato com aquela palavra.

Durante a execução, é possível ver o campo da palavra substituído por uma tabela com cada caractere e seu índice, destacando o último carácter lido para melhor compreensão, como demonstrado na Figura 5a. Além disso, no painel direito, é possível visualizar cada uma das etapas de execução, chamadas aqui de snapshots, onde aquelas que acabaram de ler o carácter destacado na tabela são destacadas em verde, conforme a Figura 5b. Em cada snapshot, é possível ver três informações: primeiro, o estado em que ela se encontra no autômato; segundo, o índice do caractere lido na palavra, indicado na tabela; e, por último, os três últimos elementos da pilha desse snapshot.

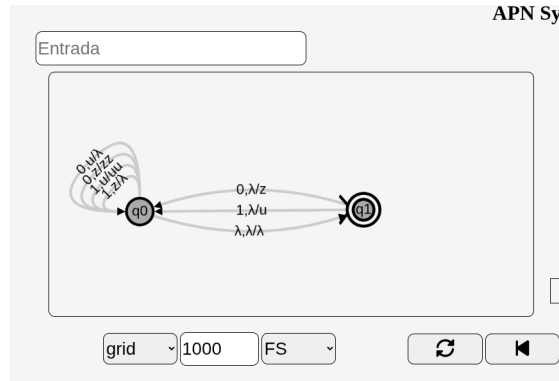


Figura 3 – Um APN simples

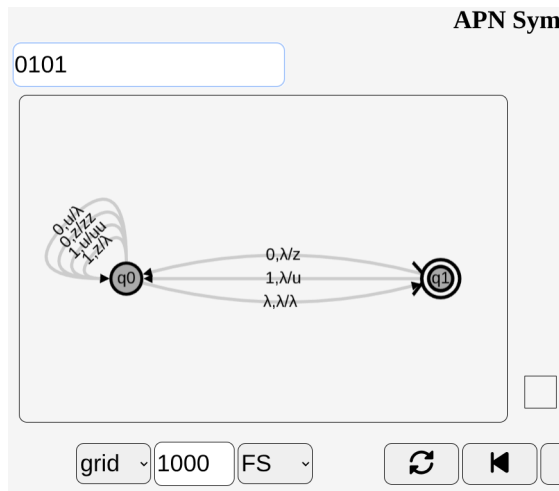


Figura 4 – entrada de palavra 0101

No painel direito, as arestas entre snapshots pintadas em azul são aquelas que podem levar a um estado de aceitação conforme mostrado na Figura 6a, e as arestas cinzas são as que não levam a um estado de aceitação. Ao clicar em um snapshot, é possível ver sua pilha completa no meio da tela, entre os dois painéis, e ver o estado atual destacado no autômato desenhado no painel esquerdo, conforme a Figura 6b. Ao clicar no botão com ícone de próximo à tabela da palavra, avança em um caractere, e os snapshots correspondentes são marcados em verde, conforme a Figura 6c, de forma semelhante. Ao clicar no botão com ícone de anterior, é visto o passo anterior de execução.

No canto inferior esquerdo, existem 3 botões como mostrado na Figura 7a: o primeiro altera o formato do desenho do APN para o padrão selecionado; o segundo é o limite de loops de execução que esse autômato executa antes de ser forçado a parar; e, por último, o critério de aceitação: FS (estado final e pilha vazia), S (pilha vazia), F (estado final). Por fim, como exibido na Figura 7b, existem dois botões no canto inferior direito: o primeiro, com ícone de disquete, serve para fazer download do autômato criado na ferramenta em formato JSON, e o segundo botão, com ícone de seta para cima, serve para fazer upload de um autômato em formato JSON presente no computador do usuário.

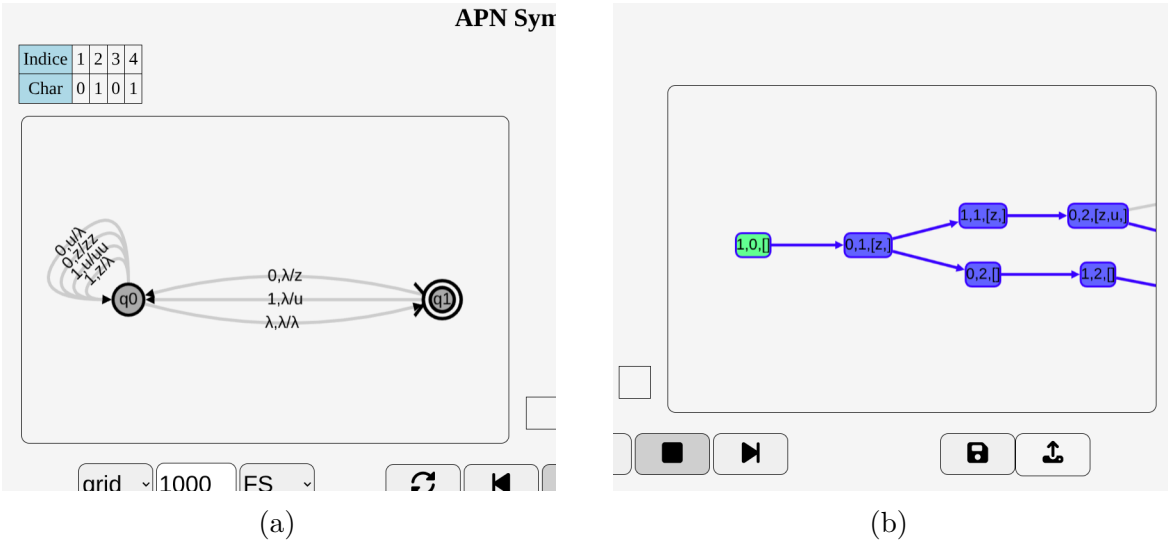
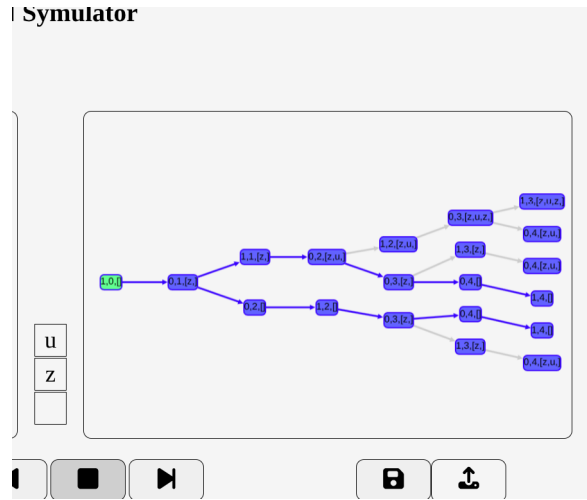
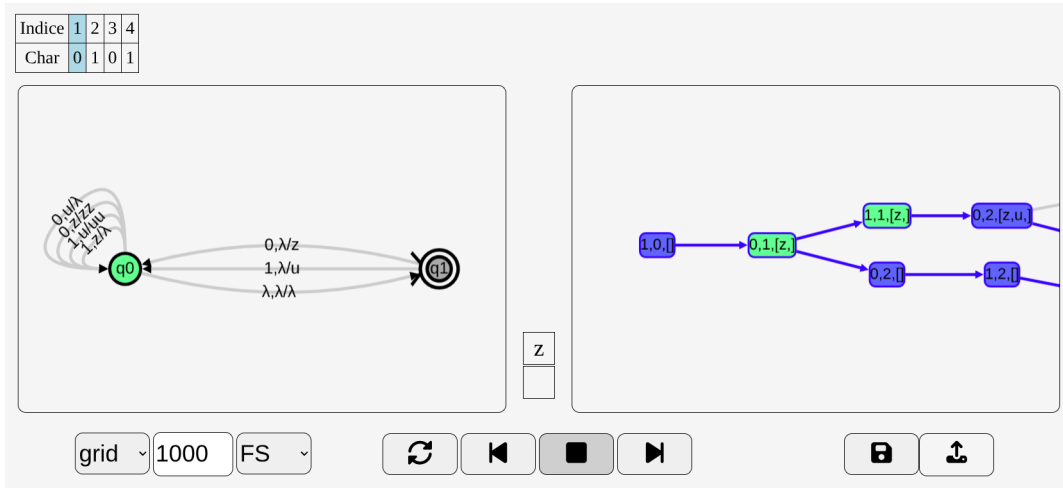


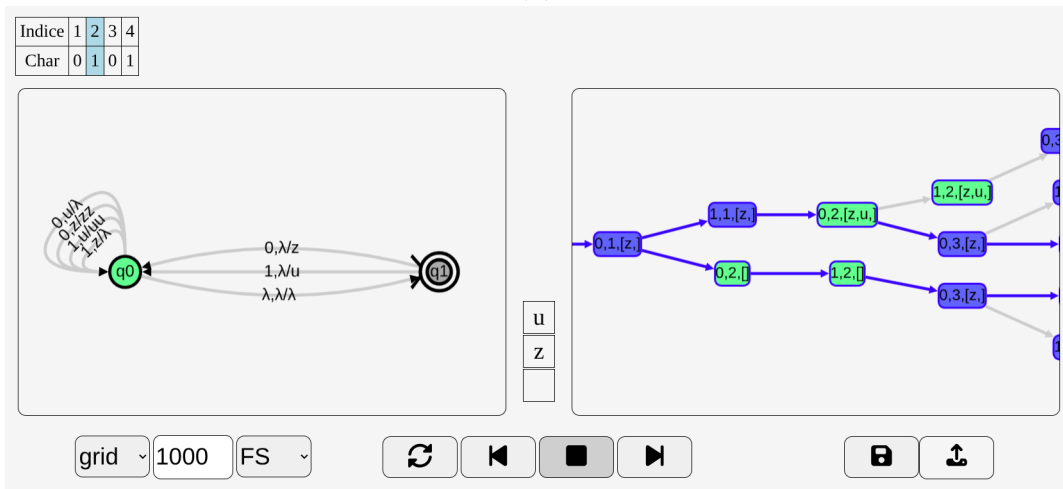
Figura 5 – a tabela de caracteres, b Visualização dos snapshots



(a)

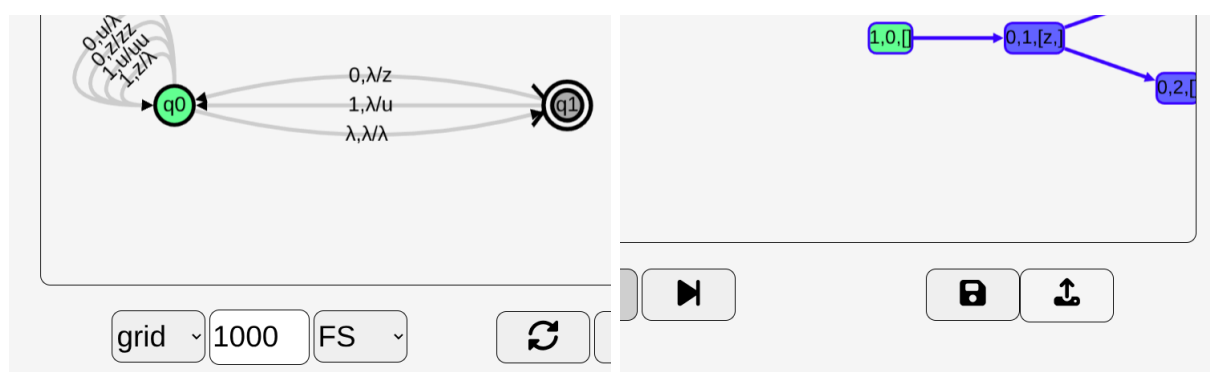


(b)



(c)

Figura 6 – a caminho de aceitação, b clique em snapshot, c botão de proximo



(a)

(b)

Figura 7 – [a](#) botões de configuração, [b](#) botões de download

4 Resultados

Este capítulo descreve os resultados finais do desenvolvimento da ferramenta proposta, abordando o que foi feito, as limitações da ferramenta desenvolvida, assim como possíveis trabalhos futuros e expansões do projeto.

Foi desenvolvido um construtor gráfico de APNs baseado em grafos que podem ser moldados pelo usuário, além de um painel que ilustra a execução de uma determinada palavra no APN e dos vários caminhos possíveis tomados pelo não determinismo. Esse painel também marca zero ou mais caminhos que resultem em aceitação. Ao executar uma determinada palavra no autômato, o usuário pode escolher entre os três tipos de aceitação: pilha vazia (S), estado final (F) ou ambos (FS). O desenvolvimento da interface foi realizado pensando em um uso mais rápido e menos verboso, em detrimento de opções mais detalhadas, focando principalmente em precisar de um menor número de cliques na tela para executar cada ação.

Os APNs construídos dentro da ferramenta podem ser baixados em formato JSON e podem ser importados de arquivos do computador do usuário para continuar com estudos anteriores. O sistema foi criado para fins gráficos e didáticos, portanto é focado em pequenos autômatos de pilha e pode apresentar lentidão dependendo do número de não determinismos e do tamanho da palavra analisada.

O sistema não foi projetado para teste em massa de autômatos grandes, mas pode ter sua eficiência refinada para melhores desempenhos em casos mais exigentes. Também não consegue evitar loops infinitos dentro do autômato, por isso existe um modificador de máximo de loops de iteração que pode ser alterado pelo usuário, pois impedir totalmente loops infinitos em APNs ainda não tem uma resolução efetiva ou não foi encontrada durante a revisão para este projeto. Navegadores modernos, no momento, utilizam apenas uma thread para sua interface gráfica ou frontend, e o projeto não utiliza backend, uma vez que uma das principais ideias era manter esse sistema hospedado de forma gratuita e estável, e um backend dificultaria essa meta. Por esse motivo, o projeto todo é executado em uma única thread, mesmo para calcular os não determinismos.

O desenvolvimento do software para outras máquinas de estado que não APNs chegou a ser iniciado, porém, devido ao tempo reduzido, foi feita a escolha de reduzir o escopo do projeto. Em trabalhos futuros, é possível finalizar a implementação das demais máquinas de estados. Além disso, não foi realizado um teste com alunos para saber sobre a real efetividade ou eficiência da ferramenta no ensino didático, portanto, esse seria um dos principais tópicos abordados em trabalhos futuros.

5 Conclusão

Este trabalho teve o objetivo de desenvolver uma ferramenta para auxiliar os alunos dos cursos da área de computação no aprendizado de linguagens formais e máquinas de estados, focando principalmente em autômatos de pilha não determinísticos (APNs), dado que esses conhecimentos são fundamentais, porém complexos e abstratos.

Entre as principais contribuições do projeto desenvolvido estão: a criação de APNs; a visualização dos múltiplos caminhos de execução de um APN passo a passo; a criação de correção de exercícios; tudo em uma plataforma completamente online, sem necessidade de instalação no computador.

Apesar disso, o sistema possui algumas limitações, sendo a execução de autômatos que podem criar um número exponencial de múltiplos caminhos quando computados com palavras grandes demais. Essa limitação varia muito de acordo com o hardware do usuário, já que é executado completamente em frontend, porém isso teoricamente não chega a ser um grande problema, já que o sistema foi desenvolvido pensando em executar autômatos voltados a exemplos didáticos e não em simulações de grande porte para pesquisas.

Dessa forma, conclui-se que a ferramenta desenvolvida demonstra potencial de auxiliar os alunos no melhor entendimento acerca de autômatos de pilha não determinísticos, porém não foi realizada pesquisa com os alunos para determinar sua eficiência ou efetividade, sendo esse um dos principais tópicos a serem abordados em trabalhos futuros, juntamente com a implementação de outras máquinas de estados além dos APNs e, talvez, o desenvolvimento de um backend para o sistema, que poderia melhorar a performance do sistema e permitir a execução de autômatos maiores.

Referências

- BURCH, C. *Autosim*. 2008. <<https://cburch.com/proj/autosim/index.html>>. Acessado em: 24 jun. 2025. Citado na página 18.
- DICKERSON, K. *Automaton Simulator*. 2021. <<https://automatonsimulator.com/>>. Acessado em: 24 jun. 2025. Citado na página 18.
- DOTY, D. *UC Davis Automaton Simulator*. 2020. <<https://web.cs.ucdavis.edu/~doty/automata/>>. Acessado em: 24 jun. 2025. Citado na página 18.
- JUKEMURA, A. S.; NASCIMENTO, H. A. D. do; UCHÔA, J. Q. Gam - um simulador para auxiliar o ensino de linguagens formais e de autômatos. *25º Congresso da Sociedade Brasileira de Computação*, 2005. Acessado em: 24 jun. 2025. Disponível em: <https://www.academia.edu/download/69969649/GAM-Um_Simulador_para_Auxiliar_o_Ensino_20210920-880-j6ritm.pdf>. Citado na página 17.
- KIENETZ, E. B.; CANAL, A. P. Simulador de autômatos finitos determinísticos - automatograph. *Disc. Scientia. Série: Ciências Naturais e Tecnológicas*, v. 5, n. 1, p. 1–9, 2004. ISSN 1519-0625. Trabalho de Iniciação Científica - PROBIC. Acessado em: 24 jun. 2025. Disponível em: <<https://periodicos.ufn.edu.br/index.php/disciplinarumNT/article/download/1176/1113>>. Citado na página 17.
- RENSSELAER; UNIVERSITY, D. *JFLAP*. 2018. <<https://www.jflap.org/>>. Acessado em: 24 jun. 2025. Citado na página 18.
- VIEIRA, L. F. M.; VIEIRA, M. A. M.; VIEIRA, N. J. Language emulator, uma ferramenta de auxílio no ensino de teoria da computação. *Anais do Congresso*, 2003. Departamento de Ciência da Computação, UFMG. Acessado em: 24 jun. 2025. Disponível em: <<http://homepages.dcc.ufmg.br/~mmvieira/publications/Wei-languageEmulator.pdf>>. Citado na página 17.
- VIEIRA, N. *Introdução aos fundamentos da computação: linguagens e máquinas*. Pioneira Thomson Learning, 2006. ISBN 9788522105083. Disponível em: <<https://books.google.com.br/books?id=62ieMQAACAAJ>>. Citado na página 22.
- WHITE, T. M.; WAY, T. P. jfast: a java finite automata simulator. *SIGCSE Bull.*, Association for Computing Machinery, New York, NY, USA, v. 38, n. 1, p. 384–388, mar. 2006. ISSN 0097-8418. Disponível em: <<https://doi.org/10.1145/1124706.1121460>>. Citado na página 18.
- ZUZAK, I.; JANKOVIC, V. *FSM Simulator*. 2025. <https://ivanzuzak.info/noam/webapps/fsm_simulator/>. Acessado em: 24 jun. 2025. Citado na página 17.